JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class SplittableRandom

java.lang.Object
    java.util.SplittableRandom

All Implemented Interfaces:

RandomGenerator, RandomGenerator.SplittableGenerator, RandomGenerator.StreamableGenerator

---

```
public final class SplittableRandom
extends Object
implements RandomGenerator, RandomGenerator.SplittableGenerator
```

A generator of uniform pseudorandom values (with period 2^{64}) applicable for use in (among other contexts) isolated parallel computations that may generate subtasks. Class `SplittableRandom` supports methods for producing pseudorandom numbers of type `int`, `long`, and `double` with similar usages as for class `Random` but differs in the following ways:

- Series of generated values pass the DieHarder suite testing independence and uniformity properties of random number generators. (Most recently validated with version 3.31.1.) These tests validate only the methods for certain types and ranges, but similar properties are expected to hold, at least approximately, for others as well. The *period* (length of any series of generated values before it repeats) is 2^{64}.
- Method `split()` constructs and returns a new SplittableRandom instance that shares no mutable state with the current instance. However, with very high probability, the values collectively generated by the two objects have the same statistical properties as if the same quantity of values were generated by a single thread using a single `SplittableRandom` object.
- Instances of SplittableRandom are *not* thread-safe. They are designed to be split, not shared, across threads. For example, a `fork/join-style` computation using random numbers might include a construction of the form `new Subtask(aSplittableRandom.split()).fork()`.
- This class provides additional methods for generating random streams, that employ the above techniques when used in `stream.parallel()` mode.

Instances of `SplittableRandom` are not cryptographically secure. Consider instead using `SecureRandom` in security-sensitive applications. Additionally, default-constructed instances do not use a cryptographically random seed unless the system property `java.util.secureRandomSeed` is set to `true`.

Since:

1.8

Nested Class Summary

Nested classes/interfaces declared in interface RandomGenerator
RandomGenerator.ArbitrarilyJumpableGenerator, RandomGenerator.JumpableGenerator, RandomGenerator.LeapableGenerator, RandomGenerator.SplittableGenerator, RandomGenerator.StreamableGenerator

Constructor Summary

Constructors

Constructor	Description
`SplittableRandom()`	Creates a new SplittableRandom instance that is likely to generate sequences of values that are statistically independent of those of any other instances in the current program; and may, and typically does, vary across program invocations.
`SplittableRandom(long seed)`	Creates a new SplittableRandom instance using the specified initial seed.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
DoubleStream	`doubles()`	Returns an effectively unlimited stream of pseudorandom `double` values from this generator and/or one split from it; each value is between zero

		(inclusive) and one (exclusive).
DoubleStream	**doubles**(double randomNumberOrig double randomNumberBound	Returns an effectively unlimited stream of pseudorandom double values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
DoubleStream	**doubles**(long streamSize)	Returns a stream producing the given streamSize number of pseudorandom double values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).
DoubleStream	**doubles**(long streamSize, double randomNumberOrigi double randomNumberBound	Returns a stream producing the given streamSize number of pseudorandom double values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
IntStream	**ints**()	Returns an effectively unlimited stream of pseudorandom int

		values from this generator and/or one split from it.
IntStream	**ints**(int randomNumberOrigin, int randomNumberBound)	Returns an effectively unlimited stream of pseudorandom int values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
IntStream	**ints**(long streamSize)	Returns a stream producing the given streamSize number of pseudorandom int values from this generator and/or one split from it.
IntStream	**ints**(long streamSize, int randomNumberOrigin, int randomNumberBound)	Returns a stream producing the given streamSize number of pseudorandom int values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
LongStream	**longs**()	Returns an effectively unlimited stream of pseudorandom long values from this generator and/or one split from it.
LongStream	**longs**(long streamSize)	Returns a stream producing the given

		streamSize number of pseudorandom long values from this generator and/or one split from it.
LongStream	**longs**(long randomNumberOrigin, long randomNumberBound)	Returns an effectively unlimited stream of pseudorandom long values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
LongStream	**longs**(long streamSize, long randomNumberOrigin, long randomNumberBound)	Returns a stream producing the given streamSize number of pseudorandom long values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).
void	**nextBytes**(byte[] bytes)	Fills a user-supplied byte array with generated byte values pseudorandomly chosen uniformly from the range of values between -128 (inclusive) and 127 (inclusive).
long	**nextLong**()	Returns a pseudorandomly chosen long value.
SplittableRandom	**split**()	Constructs and returns a new SplittableRandom instance that shares no

		mutable state with this instance.
SplittableRandom	**split**(RandomGenerator.Splitta	Returns a new pseudorandom number generator, split off from this one, that implements the RandomGenerator and RandomGenerator.SplittableGenerator interfaces.
Stream<RandomGenerator.Spl	**splits**()	Returns an effectively unlimited stream of new pseudorandom number generators, each of which implements the RandomGenerator.SplittableGenerator interface.
Stream<RandomGenerator.Spl	**splits**(long streamSize)	Returns a stream producing the given `streamSize` number of new pseudorandom number generators, each of which implements the RandomGenerator.SplittableGenerator interface.
Stream<RandomGenerator.Spl	**splits**(long streamSize, RandomGenerator.Splittab	Returns a stream producing the given `streamSize` number of new pseudorandom number generators, each of which implements the RandomGenerator.SplittableGenerator interface.
Stream<RandomGenerator.Spl	**splits**(RandomGenerator.Splitta	Returns an effectively unlimited stream of new pseudorandom number generators, each of

	which implements the RandomGenerator.SplittableGenerator interface.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Methods declared in interface RandomGenerator

equiDoubles, isDeprecated, nextBoolean, nextDouble, nextDouble, nextDouble, nextExponential, nextFloat, nextFloat, nextFloat, nextGaussian, nextGaussian, nextInt, nextInt, nextInt, nextLong, nextLong

Methods declared in interface RandomGenerator.SplittableGenerator

rngs, rngs

Constructor Details

SplittableRandom

```
public SplittableRandom(long seed)
```

Creates a new SplittableRandom instance using the specified initial seed. SplittableRandom instances created with the same seed in the same program generate identical sequences of values.

Parameters:

seed - the initial seed

SplittableRandom

```
public SplittableRandom()
```

Creates a new SplittableRandom instance that is likely to generate sequences of values that are statistically independent of those of any other instances in the current program; and may, and typically does, vary across program invocations.

Method Details

split

`public SplittableRandom split()`

Constructs and returns a new SplittableRandom instance that shares no mutable state with this instance. However, with very high probability, the set of values collectively generated by the two objects has the same statistical properties as if the same quantity of values were generated by a single thread using a single SplittableRandom object. Either or both of the two objects may be further split using the `split()` method, and the same expected statistical properties apply to the entire set of generators constructed by such recursive splitting.

Specified by:

`split` in interface `RandomGenerator.SplittableGenerator`

Returns:

the new SplittableRandom instance

split

`public SplittableRandom split (RandomGenerator.SplittableGenerator source)`

Returns a new pseudorandom number generator, split off from this one, that implements the `RandomGenerator` and `RandomGenerator.SplittableGenerator` interfaces.

Specified by:

`split` in interface `RandomGenerator.SplittableGenerator`

Parameters:

`source` - a `RandomGenerator.SplittableGenerator` instance to be used instead of this one as a source of pseudorandom bits used to initialize the state of the new ones.

Returns:

an object that implements the `RandomGenerator` and `RandomGenerator.SplittableGenerator` interfaces

Throws:

`NullPointerException` - if source is null

Since:

17

nextLong

```
public long nextLong()
```

Description copied from interface: `RandomGenerator`

Returns a pseudorandomly chosen `long` value.

Specified by:

`nextLong` in interface `RandomGenerator`

Returns:

a pseudorandomly chosen `long` value

nextBytes

```
public void nextBytes(byte[] bytes)
```

Fills a user-supplied byte array with generated byte values pseudorandomly chosen uniformly from the range of values between -128 (inclusive) and 127 (inclusive).

Specified by:

`nextBytes` in interface `RandomGenerator`

Parameters:

`bytes` - the byte array to fill with pseudorandom bytes

Throws:

`NullPointerException` - if bytes is null

Since:

10

splits

```
public Stream<RandomGenerator.SplittableGenerator> splits()
```

Returns an effectively unlimited stream of new pseudorandom number generators, each of which implements the `RandomGenerator.SplittableGenerator` interface.

This pseudorandom number generator may be used as a source of pseudorandom bits used to initialize the state the new ones.

Specified by:

`splits` in interface `RandomGenerator.SplittableGenerator`

Implementation Requirements:

The default implementation invokes `splits(this)`.

Returns:

a stream of RandomGenerator.SplittableGenerator objects

Since:

17

splits

```
public Stream<RandomGenerator.SplittableGenerator> splits
(long streamSize)
```

Returns a stream producing the given `streamSize` number of new pseudorandom number generators, each of which implements the RandomGenerator.SplittableGenerator interface.

This pseudorandom number generator may be used as a source of pseudorandom bits used to initialize the state the new ones.

Specified by:

splits in interface RandomGenerator.SplittableGenerator

Implementation Requirements:

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of RandomGenerator.SplittableGenerator objects

Throws:

IllegalArgumentException - if `streamSize` is less than zero

Since:

17

splits

```
public Stream<RandomGenerator.SplittableGenerator> splits
(RandomGenerator.SplittableGenerator source)
```

Returns an effectively unlimited stream of new pseudorandom number generators, each of which implements the RandomGenerator.SplittableGenerator interface.

Specified by:

splits in interface RandomGenerator.SplittableGenerator

Implementation Requirements:

Parameters:

`source` - a `RandomGenerator.SplittableGenerator` instance to be used instead of this one as a source of pseudorandom bits used to initialize the state of the new ones.

Returns:

a stream of `RandomGenerator.SplittableGenerator` objects

Throws:

`NullPointerException` - if source is null

Since:

17

splits

```
public Stream<RandomGenerator.SplittableGenerator> splits
(long streamSize,
 RandomGenerator.SplittableGenerator source)
```

Returns a stream producing the given `streamSize` number of new pseudorandom number generators, each of which implements the `RandomGenerator.SplittableGenerator` interface.

Specified by:

`splits` in interface `RandomGenerator.SplittableGenerator`

Implementation Requirements:

Parameters:

`streamSize` - the number of values to generate

`source` - a `RandomGenerator.SplittableGenerator` instance to be used instead of this one as a source of pseudorandom bits used to initialize the state of the new ones.

Returns:

a stream of `RandomGenerator.SplittableGenerator` objects

Throws:

`NullPointerException` - if source is null

`IllegalArgumentException` - if `streamSize` is less than zero

Since:

17

ints

```
public IntStream ints(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `int` values from this generator and/or one split from it.

Specified by:

`ints` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of pseudorandom `int` values

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero

ints

```
public IntStream ints()
```

Returns an effectively unlimited stream of pseudorandom `int` values from this generator and/or one split from it.

Specified by:

`ints` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom `int` values

ints

```
public IntStream ints(long streamSize,
                      int randomNumberOrigin,
                      int randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`ints` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `int` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

ints

```
public IntStream ints(int randomNumberOrigin,
                      int randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `int` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`ints` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `ints(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `int` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `randomNumberOrigin` is greater than or equal to `randomNumberBound`

longs

```
public LongStream longs(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it.

Specified by:

`longs` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of pseudorandom `long` values

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero

longs

```
public LongStream longs()
```

Returns an effectively unlimited stream of pseudorandom `long` values from this generator and/or one split from it.

Specified by:

`longs` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `longs(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom `long` values

longs

```
public LongStream longs(long streamSize,
                        long randomNumberOrigin,
                        long randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`longs` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `long` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

longs

```
public LongStream longs(long randomNumberOrigin,
                        long randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `long` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`longs` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `longs(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `long` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `randomNumberOrigin` is greater than or equal to `randomNumberBound`

doubles

```
public DoubleStream doubles(long streamSize)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).

Specified by:

`doubles` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

Returns:

a stream of `double` values

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero

doubles

```
public DoubleStream doubles()
```

Returns an effectively unlimited stream of pseudorandom `double` values from this generator and/or one split from it; each value is between zero (inclusive) and one (exclusive).

Specified by:

`doubles` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `doubles(Long.MAX_VALUE)`.

Returns:

a stream of pseudorandom `double` values

doubles

```
public DoubleStream doubles(long streamSize,
                            double randomNumberOrigin,
                            double randomNumberBound)
```

Returns a stream producing the given `streamSize` number of pseudorandom `double` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`doubles` in interface `RandomGenerator`

Parameters:

`streamSize` - the number of values to generate

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `double` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `streamSize` is less than zero, or `randomNumberOrigin` is not finite, or `randomNumberBound` is not finite, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

doubles

```
public DoubleStream doubles(double randomNumberOrigin,
                            double randomNumberBound)
```

Returns an effectively unlimited stream of pseudorandom `double` values from this generator and/or one split from it; each value conforms to the given origin (inclusive) and bound (exclusive).

Specified by:

`doubles` in interface `RandomGenerator`

Implementation Note:

This method is implemented to be equivalent to `doubles(Long.MAX_VALUE, randomNumberOrigin, randomNumberBound)`.

Parameters:

`randomNumberOrigin` - the origin (inclusive) of each random value

`randomNumberBound` - the bound (exclusive) of each random value

Returns:

a stream of pseudorandom `double` values, each with the given origin (inclusive) and bound (exclusive)

Throws:

`IllegalArgumentException` - if `randomNumberOrigin` is not finite, or `randomNumberBound` is not finite, or `randomNumberOrigin` is greater than or equal to `randomNumberBound`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.